ROBOTS MÓVILES AUTÓNOMOS

Ingeniería Matemática e Inteligencia Artificial

Práctica 3: Localización: Filtro de partículas

Informe de Laboratorio

Autores:
Liam Esgueva González
Jorge Carnicero Príncipe
Andrés Gil Vicente

Fecha de entrega:
16/03/2026

Índice

1. Lee el maldito manual **2**

2. ¿Perdido o encontrado? **3**

3. No tengo ni idea de dónde estoy **6**

4. Las partículas salen de excursión... y se chocan **7**

5. Echando un vistazo virtual alrededor **9**

6. Estoy de acuerdo en estar en desacuerdo **9**

7. ¿Crees lo que ven tus ojos? **10**

8. Solo sobreviven los más aptos **11**

9. La unión hace la fuerza **11**

10. Análisis de sensibilidad **13**

11. Arre, so, arre, so: Adaptación al robot real **14**

12. Uso auxiliar de Inteligencia Artificial **19**

1. Lee el maldito manual

Pregunta #1. ¿Qué métodos de maps.py pueden ser útiles para completar el código solicitado en la Sección 3? ¿Para qué los utilizaría?

Los métodos de `maps.py` más útiles para completar el código de la Sección 3 parecen ser principalmente `bounds()`, `contains()` y `check_collision()`, y de forma secundaria `plot()`. Su utilidad dentro del filtro de partículas es la siguiente:

- **`bounds()`**: permite obtener los límites geométricos del mapa. Es especialmente útil en `_init_particles`, ya que proporciona el rectángulo dentro del cual se pueden muestrear posiciones candidatas para las partículas durante la localización global. De este modo, en vez de generar posiciones arbitrarias, se restringe el muestreo a una región razonable del entorno.
- **`contains(point)`**: sirve para comprobar si una partícula está en una posición válida del mapa, es decir, en espacio libre y alcanzable.
- **`check_collision(segment)`**: puede que sea el método más importante para las secciones 4 y 5. En primer lugar, durante la **predicción del movimiento**, permite detectar si el desplazamiento de una partícula desde su posición anterior hasta la nueva cruza una pared; en tal caso, la partícula puede recolocarse en el primer punto de intersección para evitar que atraviese obstáculos. En segundo lugar, durante la **simulación del LiDAR** en `_sense`, este mismo método se usa para calcular dónde impacta cada rayo virtual con el entorno y obtener así la distancia predicha de cada haz.
- **`plot()`**: Nos permite plotear el mapa y superponer sobre él las partículas en cada iteración.

En resumen, `bounds()` y `contains()` son esenciales para generar partículas iniciales válidas, mientras que `check_collision()` es la pieza clave tanto para impedir que las partículas generadas atraviesen paredes como para simular las medidas del sensor LiDAR.

Pregunta #2. ¿Qué optimizaciones se han introducido en la clase Map para reducir los tiempos de ejecución de las Secciones 4 y 5?

La clase `Map` incorpora varias optimizaciones orientadas a reducir el coste computacional de las secciones 4 y 5, que son precisamente las más caras del filtro de partículas porque exigen realizar muchas comprobaciones geométricas.

- **Engrosamiento automático de obstáculos mediante `safety_distance`**. La clase no trabaja únicamente con las paredes "ideales", sino con una versión engrosada de los

obstáculos que ya tiene en cuenta el tamaño del robot. Gracias a ello, al comprobar si una partícula es válida o si un rayo colisiona, no hace falta recalcular continuamente la geometría real del robot frente a cada pared. Esto simplifica mucho los cálculos y evita operaciones geométricas más complejas.

- **Intersecciones compiladas (`compiled_intersect=True`)**. En la creación del mapa se activa explícitamente esta opción:

  ```
  Map(map_path, sensor_range_max, compiled_intersect=True,
          use_regions=False, safety_distance=0.08)
  ```

  Esto indica que el cálculo de intersecciones segmento–pared está optimizado internamente, algo especialmente importante porque `check_collision()` se invoca muchísimas veces: una vez por partícula al moverla, y varias veces más por partícula al simular los rayos del LiDAR.

- **Uso opcional de regiones (`use_regions`)**. Cuando este booleano está activado y se reduce el rango del LiDAR, `check_collision()` puede descartar automáticamente las paredes demasiado lejanas y comprobar solo una fracción del mapa. Esto reduce notablemente el número de segmentos con los que hay que intersecar cada rayo.

- **Representación preprocesada del espacio libre**. La clase `Map` ya deja preparado el entorno para consultas rápidas como `contains()` o `check_collision()`, de modo que el filtro de partículas no tiene que reconstruir ni reinterpretar el mapa en cada iteración.

Estas optimizaciones son relevantes sobre todo en las secciones 4 y 5 porque ahí se concentra la mayor parte del coste: en la sección 4 se comprueba si cada partícula ha atravesado paredes tras moverse, y en la sección 5 se lanzan múltiples rayos virtuales por cada partícula para comparar medidas. Por tanto, cualquier mejora en el cálculo de colisiones e intersecciones repercute directamente en el rendimiento global del filtro.

2. ¿Perdido o encontrado?

En esta sección establecemos la comunicación básica del filtro de partículas creando el topic `/pose` para notificar si el algoritmo ha convergido. Utilizamos el tipo de mensaje personalizado `PoseStamped`.

Código 2.0: Publicación y suscripción a la pose - TODO 3.1, 3.2 y 3.3

```
# TODO: 3.1. Create the /pose publisher (PoseStamped message).
self._pose_publisher = self.create_publisher(PoseStamped, "pose", 10)

# TODO: 3.2. Complete the function body with your code (i.e., replace the
    pass statement).
```

```python
def _publish_pose_estimate(self, x_h: float, y_h: float, theta_h: float) ->
    None:

        # Create the msg
        msg = PoseStamped()
        msg.localized = self._localized  # We add this information wether is
     true or false
        msg.header.stamp = self.get_clock().now().to_msg()  # We add the
    header (stamp)

        if self._localized:  # If the robot is localized, we add the pose
    info to the msg
            msg.pose.position.x = x_h
            msg.pose.position.y = y_h

            w, x, y, z = euler2quat(0.0, 0.0, theta_h)  # Roll and pitch are
     zero. We introduce yaw
            msg.pose.orientation.w = w
            msg.pose.orientation.x = x
            msg.pose.orientation.y = y
            msg.pose.orientation.z = z

        self._pose_publisher.publish(msg)  # We publish the msg

# TODO: 3.3. Subscribe to /pose if self._enable_localization is True.
self._subscribers: list[ message_filters.Subscriber ] = []

# Append the cmd vel suscriber
self._subscribers.append(
    message_filters.Subscriber(
        self,
        TwistStamped,
        "cmd_vel",
        qos_profile = QoSProfile(depth=10)
    )
)

# Append the pose suscriber
self._subscribers.append(
    message_filters.Subscriber(
        self,
        PoseStamped,
        "pose",
        qos_profile = QoSProfile(depth=10)
    )
)
```

```
ts = message_filters.ApproximateTimeSynchronizer(
    self._subscribers,
    queue_size=10,
    slop=10.0
)
ts.registerCallback(self._next_step_callback)
```

3. No tengo ni idea de dónde estoy

El primer paso lógico del filtro de partículas es la inicialización. En este apartado hemos completado el método `_init_particles` para generar N partículas válidas.

Código 3.0: Inicialización de partículas - TODO 3.4

```
def _init_particles(
    self,
    particle_count: int,
    global_localization: bool,
    initial_pose: tuple[float, float, float],
    initial_pose_sigma: tuple[float, float, float],
) -> np.ndarray:

    particles = np.empty((particle_count, 3), dtype=float)

    # TODO: 3.4. Complete the missing function body with your code.

    # Extract the bounds of the map
    x_min, y_min, x_max, y_max = self._map.bounds()

    # Extract the values needed to create particles
    x_o, y_o, theta_o = initial_pose
    x_o_std, y_o_std, theta_o_std = initial_pose_sigma

    for i in range(particle_count):
        valid = False

        while not valid:
            if global_localization:  # First localization mode
                particle_x = np.random.uniform(low=x_min, high=x_max)
                particle_y = np.random.uniform(low=y_min, high=y_max)
                orientation = np.random.choice([0, np.pi / 2, np.pi, 3 * np.
   pi / 2])

            else:  # Pose tracking mode
                particle_x = np.random.normal(loc=x_o, scale=x_o_std)
                particle_y = np.random.normal(loc=y_o, scale=y_o_std)
                orientation = np.random.normal(loc=theta_o, scale=
   theta_o_std)

            # If it is in an invalid place, we generate it again
            if self._map.contains(
                (particle_x, particle_y)
            ):  # If the particle is valid, we store it
```

```
                particles[i] = [
                    particle_x,
                    particle_y,
                    orientation % (2 * math.pi),
                ]  # To normalize and just have values in range [0, 2*pi]
                valid = True

    return particles
```

4. Las partículas salen de excursión... y se chocan

Durante la fase de predicción del filtro de partículas, desplazamos cada partícula según los comandos de velocidad v y ω aplicando un modelo cinemático e inyectando ruido gaussiano para representar la incertidumbre estocástica del movimiento.

Código 4.0: Movimiento de las partículas con ruido - TODO 3.5

```
def move(self, v: float, w: float) -> None:
    self._iteration += 1

    # TODO: 3.5. Complete the function body with your code.

    for particle in self._particles:
        # Extract the components of this particle, to update them
        x_o, y_o, theta_o = particle

        # Generate noise
        noise_v = np.random.normal(loc=0, scale=self._sigma_v)
        noise_w = np.random.normal(loc=0, scale=self._sigma_w)

        # Proyect the velocities to the x,y axis (global world axis)
        x_vel = np.cos(theta_o) * (v + noise_v)
        y_vel = np.sin(theta_o) * (v + noise_v)

        # Update x,y,theta with noise
        particle[0] += x_vel * self._dt
        particle[1] += y_vel * self._dt
        particle[2] += (w + noise_w) * self._dt

        # Normalize to make sure that the angle is in range [0, 2*pi]
        particle[2] %= 2 * np.pi

        # Compute the intersection with the walls
```

```
        colission_segment = [(x_o, y_o), (particle[0], particle[1])]
    colissions, _ = self._map.check_collision(
        segment=colission_segment, compute_distance=True
    )

        if colissions:
            # Readjust the position of the particle
            particle[0] = colissions[0]
            particle[1] = colissions[1]
```

5. Echando un vistazo virtual alrededor

En este apartado implementamos el método `_sense`, cuyo objetivo es simular qué mediría el sensor LiDAR si el robot estuviera en la pose de una partícula concreta. Para ello generamos un conjunto de rayos virtuales a partir de dicha pose utilizando la función `_lidar_rays`.

Posteriormente comprobamos dónde colisiona cada rayo con el mapa mediante `check_collision`, lo que nos permite obtener la distancia predicha hasta el obstáculo más cercano en cada dirección. Para reducir el coste computacional, en lugar de utilizar todos los haces del LiDAR se toma únicamente un subconjunto uniformemente espaciado de rayos.

Código 5.0: Simulación de lecturas LiDAR - TODO 3.6

```python
def _sense(self, pose: tuple[float, float, float]) -> list[float]:
  # TODO: 3.6. Complete the missing function body with your code.
  rays = np.arange(0, 240, 240 // 8).tolist()

  segments = self._lidar_rays(pose=pose, indices=rays)

  z_hat: list[float] = [
      self._map.check_collision(segment=pair, compute_distance=True)[1]
      for pair in segments
  ]

  return z_hat
```

6. Estoy de acuerdo en estar en desacuerdo

Para evaluar qué tan buena es una partícula, necesitamos comparar sus medidas virtuales con las medidas reales del LiDAR. Esto lo hacemos evaluándolas en la distribución gaussiana correspondiente.

Código 6.0: Cálculo de la discrepancia Gaussiana - TODO 3.7

```python
@staticmethod
def _gaussian(mu: float, sigma: float, x: float) -> float:
  # TODO: 3.7. Complete the function body (i.e., replace the code below).
  return np.exp(-0.5 * ((x - mu) / sigma) ** 2) / (sigma * np.sqrt(2 * np.pi
    ))
```

7. ¿Crees lo que ven tus ojos?

En este apartado implementamos el método `_measurement_probability`, que calcula la verosimilitud de un conjunto de medidas LiDAR para una partícula dada. La idea consiste en comparar las lecturas reales del sensor con las medidas predichas por `_sense` para esa pose hipotética. Para ello se toma el mismo subconjunto de haces utilizado en la simulación del LiDAR y, para cada par de medidas válidas, se evalúa una probabilidad gaussiana en función de su discrepancia.

Finalmente, la probabilidad total de la partícula se obtiene como el producto de las contribuciones de todos los haces considerados, tratando aquellas lecturas no válidas identificadas con `nan`, cambiándolas por el valor del rango mínimo de medida de nuestro sensor.

Código 7.0: Cálculo de la probabilidad de las medidas - TODO 3.8

```python
def _measurement_probability(
    self, measurements: list[float], particle: tuple[float, float, float]
) -> float:
    # TODO: 3.8. Complete the missing function body with your code.

    probability = 1.0

    predicted_measurements = self._sense(pose=particle)
    rays = range(0, 240, 240 // 8)
    subsampled_measurements = [measurements[i] for i in rays]
    for measurement, predicted_measurement in zip(
        subsampled_measurements, predicted_measurements
    ):
        if np.isnan(measurement):
            measurement = self._sensor_range_min

        if np.isnan(predicted_measurement):
            predicted_measurement = self._sensor_range_min

        probability *= self._gaussian(
            mu=measurement, sigma=self._sigma_z, x=predicted_measurement
        )

    return probability
```

8. Solo sobreviven los más aptos

La fase de corrección culmina con el remuestreo (resampling).

Código 8.0: Remuestreo de partículas - TODO 3.9

```
def resample(self, measurements: list[float]) -> None:
    # TODO: 3.9. Complete the function body with your code (i.e., replace
    the pass statement).

    probabilities = np.array(
        [self._measurement_probability(measurements, particle) for particle
    in self._particles],
        dtype=float,
    )

    total = sum(probabilities)
    probabilities = probabilities / total
    n = self._particle_count
    rand_numbers = np.random.uniform(0, 1 / n) + np.arange(n) / n

    # we sum the weights in order to apply the  stratific samples
    weight_circle = np.cumsum(probabilities)

    # we choose the particles
    prominent_weights = np.digitize(rand_numbers, weight_circle)
    prominent_weights = np.clip(prominent_weights, 0, len(self._particles) -
     1)

    self._particles = self._particles[prominent_weights]

    if self._logger is not None:
        self._logger.warning(f"Ejecutando resample. N Particulas: {len(self.
    _particles)}.")
```

9. La unión hace la fuerza

En este apartado implementamos el método `compute_pose`, encargado de determinar si las partículas han convergido a una única hipótesis de localización. Para ello aplicamos el algoritmo de `DBSCAN` sobre las partículas, representando la orientación mediante seno y coseno para evitar discontinuidades angulares.

Si se detecta un único clúster, consideramos válida la localización y calculamos la pose estimada como el promedio de las partículas agrupadas. Si aparecen varios clústeres, inter-

pretamos que persisten varias hipótesis y ajustamos dinámicamente el número de partículas para conservar esa diversidad.

Código 9.0: Clustering DBSCAN y estimación de pose - TODO 3.10

```
def compute_pose(self) -> tuple[bool, tuple[float, float, float]]:
    # TODO: 3.10. Complete the missing function body with your code.

    localized = False
    pose = (float("nan"), float("nan"), float("nan"))

    particles_projected = np.array(self._particles, dtype=float)
    theta = particles_projected[:, -1]

    particles_projected = np.hstack(
        [particles_projected[:, :-1], np.cos(theta)[:, None], np.sin(theta)
    [:, None]]
    )

    clustering = DBSCAN(eps=0.2, min_samples=5).fit(particles_projected)
    labels = clustering.labels_
    n_clusters = len(set(labels) - {-1})
    indexes = clustering.core_sample_indices_

    if n_clusters == 1:
        localized = True
        cluster_particles = np.asarray(self._particles[indexes], dtype=np.
    float64)

        x_mean = np.mean(cluster_particles[:, 0])
        y_mean = np.mean(cluster_particles[:, 1])

        theta_mean = math.atan2(
            np.mean(np.sin(cluster_particles[:, -1])), np.mean(np.cos(
    cluster_particles[:, -1]))
        ) % (2 * math.pi)

        pose = (x_mean, y_mean, theta_mean)

        self._particle_count = 100

    elif n_clusters > 1:
        self._particle_count = max(int(100 * n_clusters), 100)

    return localized, pose
```

10. Análisis de sensibilidad

Pregunta #3: Efecto del número de partículas y estrategia de reducción

El número de partículas influye directamente tanto en el tiempo de computación como en la precisión de la localización. Al aumentar el número de partículas, el filtro puede representar mejor la distribución de probabilidad de la pose del robot y mantener más hipótesis simultáneamente, lo que mejora la robustez y la exactitud de la estimación. Sin embargo, esto incrementa también el coste computacional, ya que todas las operaciones del filtro (predicción, simulación del LiDAR, cálculo de probabilidades y remuestreo) se realizan para cada partícula.

Por el contrario, si el número de partículas es demasiado pequeño, el filtro pierde capacidad para explorar el espacio de estados y puede eliminar hipótesis correctas demasiado pronto, lo que puede provocar convergencias incorrectas o inestables.

En este entorno, un número inicial moderado de partículas proporciona un buen equilibrio entre precisión y eficiencia. En nuestro caso personal, este valor óptimo ha sido de 2000 partículas iniciales.

Por otro lado, la estrategia que nos ha resultado más eficaz ha consistido en comenzar con más partículas durante la localización global y reducir su número cuando las partículas se concentran en un único clúster, como hemos comentado anteriormente en la función de `compute_pose` . De este modo se mantiene la diversidad cuando hay incertidumbre y se reduce el coste computacional cuando la localización ya ha convergido. Esto concuerda con lo que hemos estudiado en la teoría de la asignatura.

Pregunta #4: Efecto de los parámetros de ruido

Los parámetros de ruido controlan la incertidumbre del modelo de movimiento y del modelo de medida. Si el ruido de movimiento (σ_v y σ_w) es demasiado pequeño, el filtro confía demasiado en el modelo cinemático y las partículas pueden desviarse de la pose real y eliminarse sin posibilidad de recuperarse. Si es demasiado grande, las partículas se dispersan en exceso y la convergencia se vuelve más lenta e inestable.

Por otro lado, el parámetro de ruido de medida (σ_z) determina cuánto se penalizan las diferencias entre las medidas reales y las predichas por el LiDAR. Valores muy pequeños hacen el filtro demasiado estricto frente a pequeñas discrepancias, mientras que valores demasiado grandes reducen su capacidad para distinguir entre hipótesis correctas e incorrectas.

Por tanto, hemos observado que es necesario elegir valores intermedios adecuados que permitan mantener diversidad en las partículas y, al mismo tiempo, aprovechar la información del sensor para favorecer las hipótesis más probables.

11. Arre, so, arre, so: Adaptación al robot real

Para resolver el problema de *sim-to-real* provocado por los elevados tiempos de cómputo del filtro de partículas implementado en Python, hemos diseñado una estrategia de sincronización explícita entre el nodo de navegación (seguidor de pared) y el de localización. A diferencia de la simulación en CoppeliaSim, el robot físico no detiene el tiempo a la espera de que se procesen las medidas. Si no intervenimos, el seguidor de pared continuaría moviendo el TurtleBot 3 basándose en información desactualizada, impidiendo por completo la convergencia del algoritmo.

Hemos implementado los siguiente puntos:

1. **Mensaje personalizado para el control de marcha:** Hemos definido un nuevo mensaje en el paquete `amr_msgs` que contiene un campo booleano. Este mensaje es el encargado de transmitir la orden de detener o reanudar el movimiento.

Código 11.0: Punto 1.1: Mensaje personalizado

```
std_msgs/Header header
bool stop_robot
```

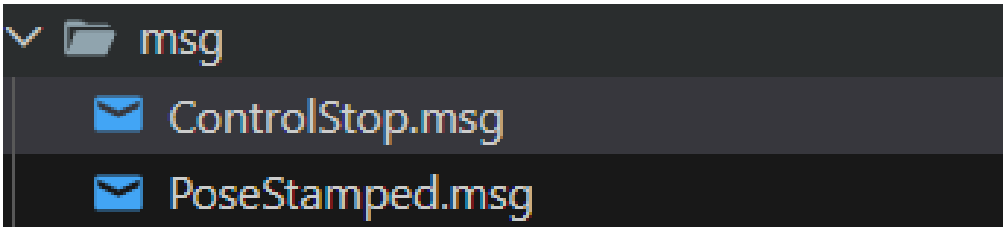

Figura 1: Mensaje personalizado creado

Código 11.1: Punto 1.2: CMakeLists.txt modificado

```
set(msg_files
  "msg/PoseStamped.msg"
  "msg/ControlStop.msg"
)
```

2. **Comunicación entre nodos:** Hemos creado un publicador en el `ParticleFilterNode` y un suscriptor independiente en el `WallFollowerNode` operando sobre un nuevo *topic* (`/stop_condition`). El seguidor de pared almacena el estado recibido. En su bucle de control, si el estado indica que debe detenerse, publica comandos de velocidad estrictamente nulos ($v = 0, \omega = 0$). Si puede seguir, ejecuta el algoritmo.

Código 11.2: Punto 2: Suscripción y Publicador

```
# 3.11.2 Create publisher for the stop condition - ParticleFilterNode
self._stop_publisher = self.create_publisher(
    ControlStop, "stop_condition", 10
)

# 3.11.2 Subscriber for the personalized topic - WallFollowerNode
self._personalized_subscriber = self.create_subscription(
    msg_type=ControlStop,
    topic="stop_condition",
    callback=self._compute_personalized_stop_callback,
    qos_profile=10
)

# 3.11.2 Function to change internal node value
def _compute_personalized_stop_callback(self, msg: ControlStop):
    self._stop_robot = msg.stop_robot

# _compute_commands_callback control
if self._stop_robot:
    self._publish_velocity_commands(0.0, 0.0)
    return
```

3. **Almacenamiento asíncrono (Buffers):** Hemos eliminado la sincronización de `message_filters`. Ahora, el callback del *LiDAR* (`/scan`) simplemente sobrescribe una variable con el escaneo más reciente. Y el callback de odometría extrae las velocidades lineal y angular (v, ω) y las añade a una lista en forma de tuplas. Para evitar sobrecargar la memoria con datos irrelevantes cuando el robot está detenido, descartamos aquellas medidas cuyas velocidades sean próximas a cero.

Código 11.3: Punto 3: Callbacks asíncronos en ParticleFilterNode

```
self._scan_subscriber = self.create_subscription(
    LaserScan, "scan", self._callback_scan_saving_history,
    scan_qos_profile
)

self._odometry_subscriber = self.create_subscription(
    Odometry, "odometry", self._callback_odometry_saving_history, 10
)
```

```python
# 3.11.3 /scan
def _callback_scan_saving_history(self,scan_msg: LaserScan):
    z_scan: list[float] = scan_msg.ranges

    self._scan_last_measures = z_scan

# 3.11.3 /odometry
def _callback_odometry_saving_history(self, odom_msg: Odometry):
    z_v: float = odom_msg.twist.twist.linear.x
    z_w: float = odom_msg.twist.twist.angular.z

    if abs(z_v) >= 1e-6 or abs(z_w) >= 1e-6:
        self._odometry_estimate_list.append((z_v, z_w))
```

4. **Temporizador de sincronización (5 segundos):** La carga computacional se confina en un temporizador dedicado en el `ParticleFilterNode` que se ejecuta cada 5 segundos. La secuencia lógica es estricta: primero publica el mensaje para detener al robot; luego, ejecuta tantas fases de predicción como tuplas haya en el *buffer* de odometría (vaciándolo a continuación); tras ello, realiza una única fase de corrección con el escaneo más reciente para estimar la pose; seguidamente, publica la orden de reanudar la marcha; y finalmente, publica la pose global estimada.

Código 11.4: Punto 4: Lógica principal del temporizador en ParticleFilterNode

```python
timer = self.create_timer(5, self._timer_callback)

def _timer_callback(self):


    # a)
    stop_msg = ControlStop()
    stop_msg.stop_robot = True
    stop_msg.header.stamp = self.get_clock().now().to_msg()
    self._stop_publisher.publish(stop_msg)

    # b)
    odometry_estimates = list(self._odometry_estimate_list)
    self._odometry_estimate_list.clear()
    for z_v, z_w in odometry_estimates:
        self._execute_motion_step(z_v, z_w)
        self._steps += 1
```

```
        # c)
        x_h, y_h, theta_h = 0.0, 0.0, 0.0
        if self._scan_last_measures:
            x_h, y_h, theta_h = self._execute_measurement_step(self.
       _scan_last_measures)
            self._scan_last_measures = []

        # d)
        stop_msg = ControlStop()
        stop_msg.stop_robot = self._localized
        stop_msg.header.stamp = self.get_clock().now().to_msg()
        self._stop_publisher.publish(stop_msg)

        # e)
        if self._localized:
            x_h, y_h, theta_h = self._particle_filter.compute_pose()[1]
            self._publish_pose_estimate(x_h, y_h, theta_h)
```

Vídeo #1

Este vídeo se adjunta en la entrega junto con el resto de códigos y materiales pedidos. Tiene el nombre de `RobotsMovilesAutonomos_P3_G21_Centre_Corridor_Video` . Se localiza partiendo del centro.

Imagenes correspondiente a la localización del robot cuando es inicializado en el pasillo central.

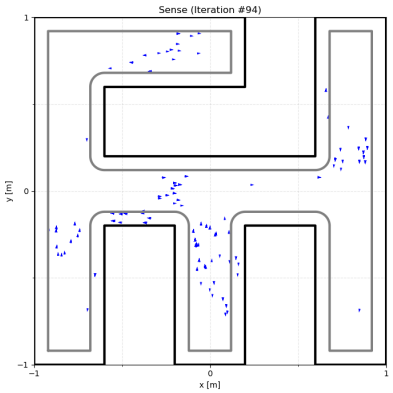

Figura 1: Inicialización

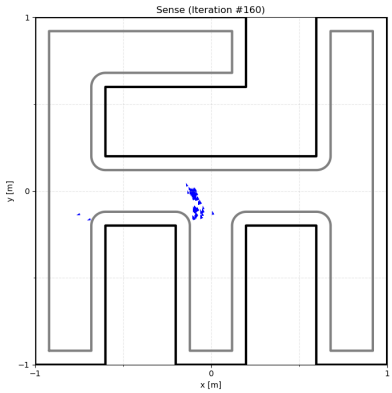

Figura 2: Segunda fase

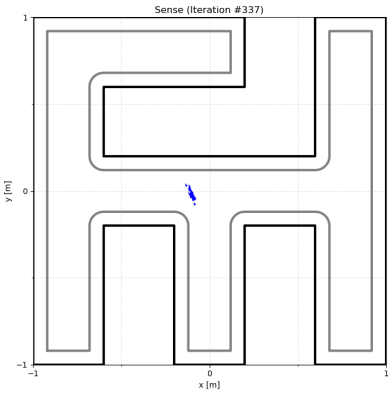

Figura 3: Tercera fase

Figura 2: Evolución de las vistas centrales desde la inicialización.

Además, en la carpeta de outputs se adjuntan todas las demás imágenes a modo de anexo.

Vídeo #2

Este vídeo se adjunta en la entrega junto con el resto de códigos y materiales pedidos. Tiene el nombre de `RobotsMovilesAutonomos_P3_G21_Corner_Corridor_Video` . Se localiza partiendo de una esquina.

Imagenes correspondiente a la localización del robot cuando es inicializado en el pasillo izquierdo.

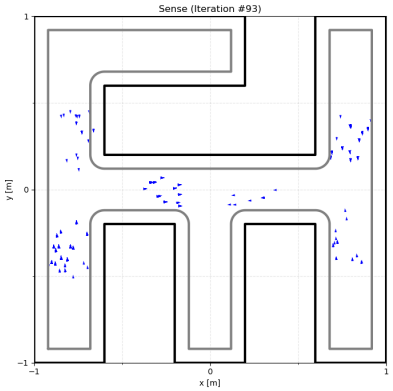

Figura 1: Primera fase

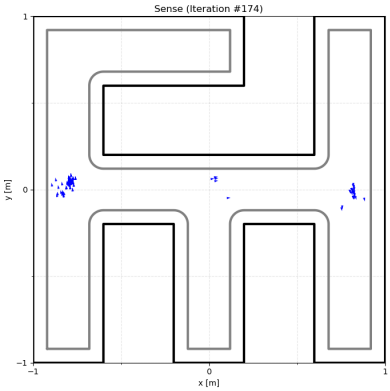

Figura 2: Segunda fase

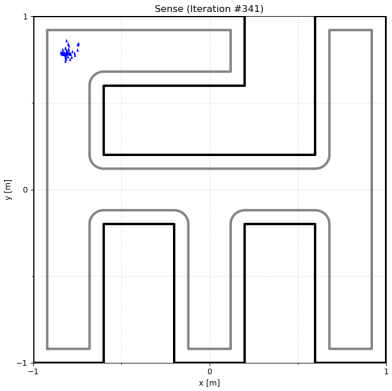

Figura 3: Tercera fase

Figura 3: Evolución de las vistas izquierdas.

Además, en la carpeta de outputs se adjuntan todas las demás imágenes a modo de anexo.

12. Uso auxiliar de Inteligencia Artificial

A lo largo de la elaboración de esta memoria se ha empleado inteligencia artificial como herramienta de apoyo, sin sustituir en ningún caso el trabajo de comprensión, experimentación y redacción realizado por los autores. En concreto, su uso se ha limitado a tareas auxiliares orientadas a mejorar la claridad y la presentación del documento:

- **Corrección lingüística:** se ha utilizado para detectar y corregir faltas de ortografía, erratas y pequeños problemas de concordancia o puntuación.
- **Mejora del estilo y uniformidad:** se ha empleado para sugerir ajustes de redacción (frases más claras, eliminación de repeticiones y homogeneización del tono).